```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (train_test_split,GridSearchCV,cross_val_score,StratifiedKFold)
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classific

import warnings
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
```

```
import os

file_path = "diabetes (6).csv"

if os.path.exists(file_path):
    df = pd.read_csv(file_path)
    print("Shape:", df.shape)
else:
    print(f"Error: The file '{file_path}' does not exist. Please upload it or provide the correct path.")
```

```
Shape: (768, 9)
```

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	3.0000	6.00000	17.00
Glucose	768.0	120.894531	31.972618	0.000	99.00000	117.0000	140.25000	199.00
BloodPressure	768.0	69.105469	19.355807	0.000	62.00000	72.0000	80.00000	122.00
SkinThickness	768.0	20.536458	15.952218	0.000	0.00000	23.0000	32.00000	99.00
Insulin	768.0	79.799479	115.244002	0.000	0.00000	30.5000	127.25000	846.00
BMI	768.0	31.992578	7.884160	0.000	27.30000	32.0000	36.60000	67.10
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.24375	0.3725	0.62625	2.42
Age	768.0	33.240885	11.760232	21.000	24.00000	29.0000	41.00000	81.00
Outcome	768.0	0.348958	0.476951	0.000	0.00000	0.0000	1.00000	1.00

```
missing = df.isnull().sum()

print("Missing vakues")
print(missing)
```

```
Missing vakues
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:",duplicate_count)
```

```
Number of duplicate rows: 0
```

```
if duplicate_count > 0:
  df = df.drop_duplicates().reset_index(drop=True)
  print(f"Removed {duplicate_count} duplicate rows. New DataFrame shape: {df.shape}")
else:
  print("No duplicate rows found.")
```

```
No duplicate rows found.
```

```
print(df["Outcome"].value_counts())
```

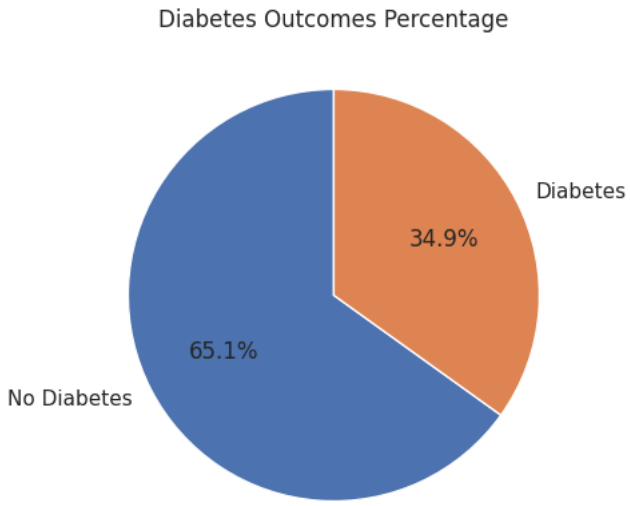
```
Outcome
0    500
1    268
Name: count, dtype: int64
```

```
target_percentage = (
    df["Outcome"] # Corrected column name to 'Outcome'
    .value_counts(normalize=True)
    .sort_index()
    * 100
)
print(target_percentage)
```

```
Outcome
0    65.104167
1    34.895833
Name: proportion, dtype: float64
```

```
plt.figure(figsize=(7,5))

plt.pie(target_percentage,labels=["No Diabetes", "Diabetes"],autopct="%1.1f%%",startangle=90
)
plt.title("Diabetes Outcomes Percentage") # Corrected typo from ply.title to plt.title
plt.show()
```

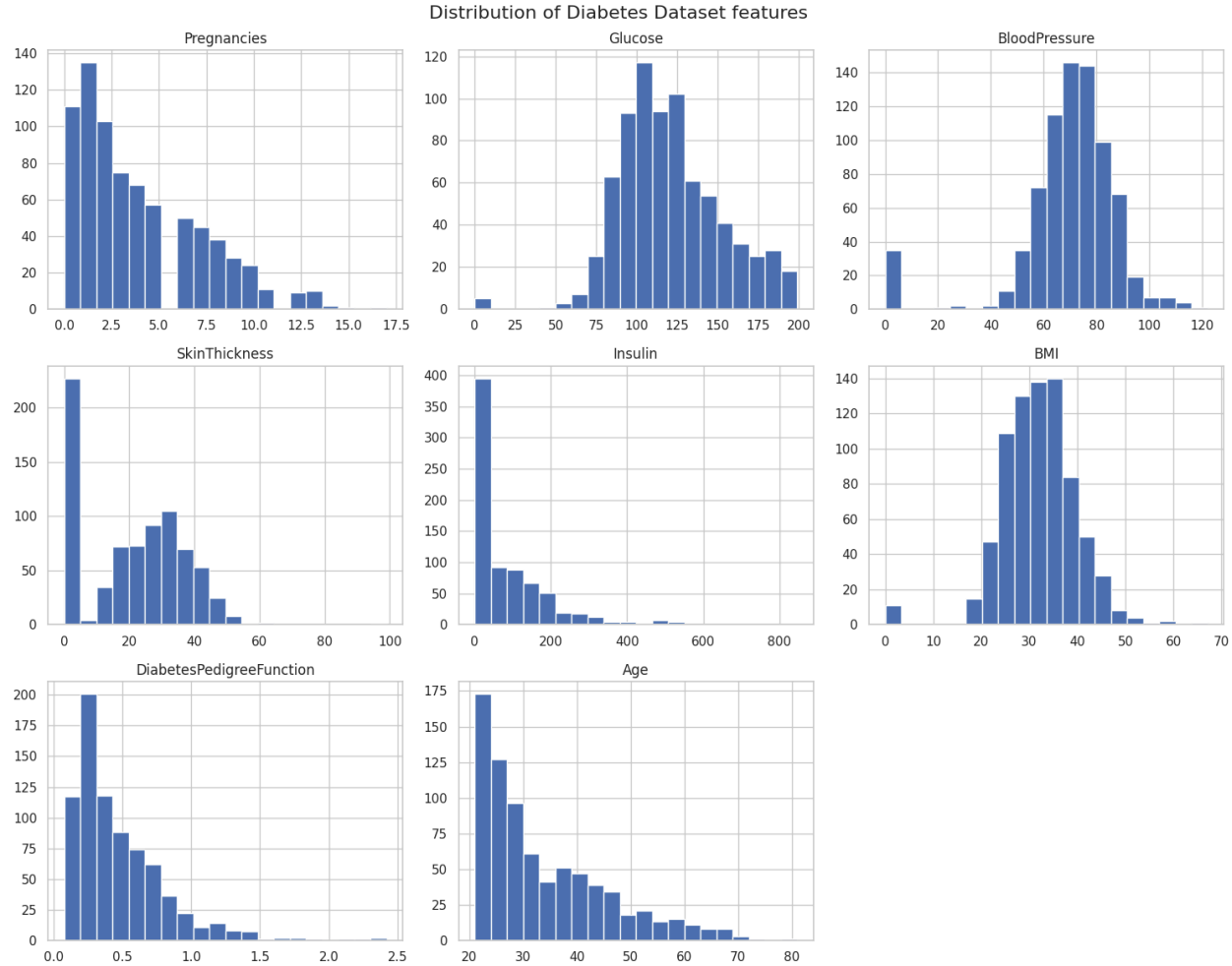



```
features = df.drop(columns="Outcome").columns

df[features].hist(figsize=(15,12),bins=20)

plt.suptitle("Distribution of Diabetes Dataset features",fontsize=16)

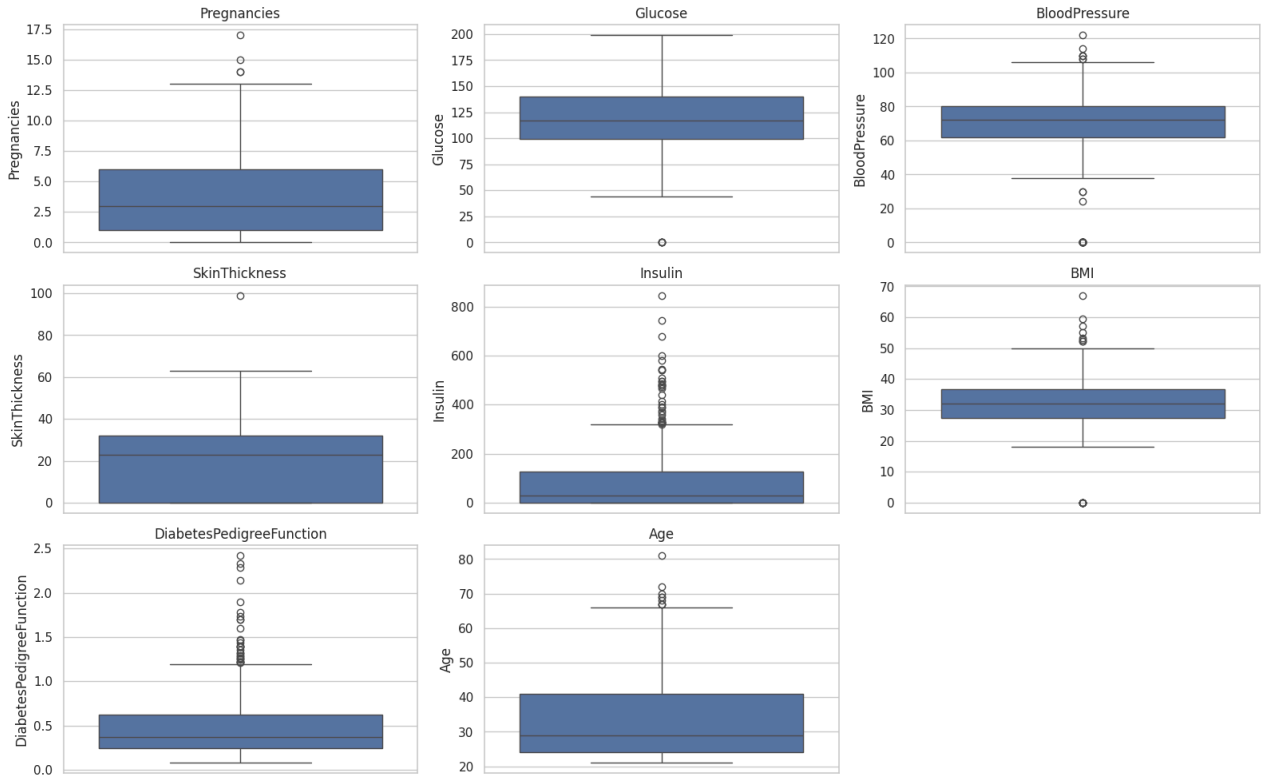
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(16,10))

for i,col in enumerate(features, 1):
  plt.subplot(3, 3, i)

  sns.boxplot(y=df[col])
  plt.title(col)
plt.tight_layout()
plt.show()
```




```
zero_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness","Insulin","BMI"
]
zero_counts = (df[zero_columns] == 0).sum()

print("Zero values:")
print(zero_counts)
```

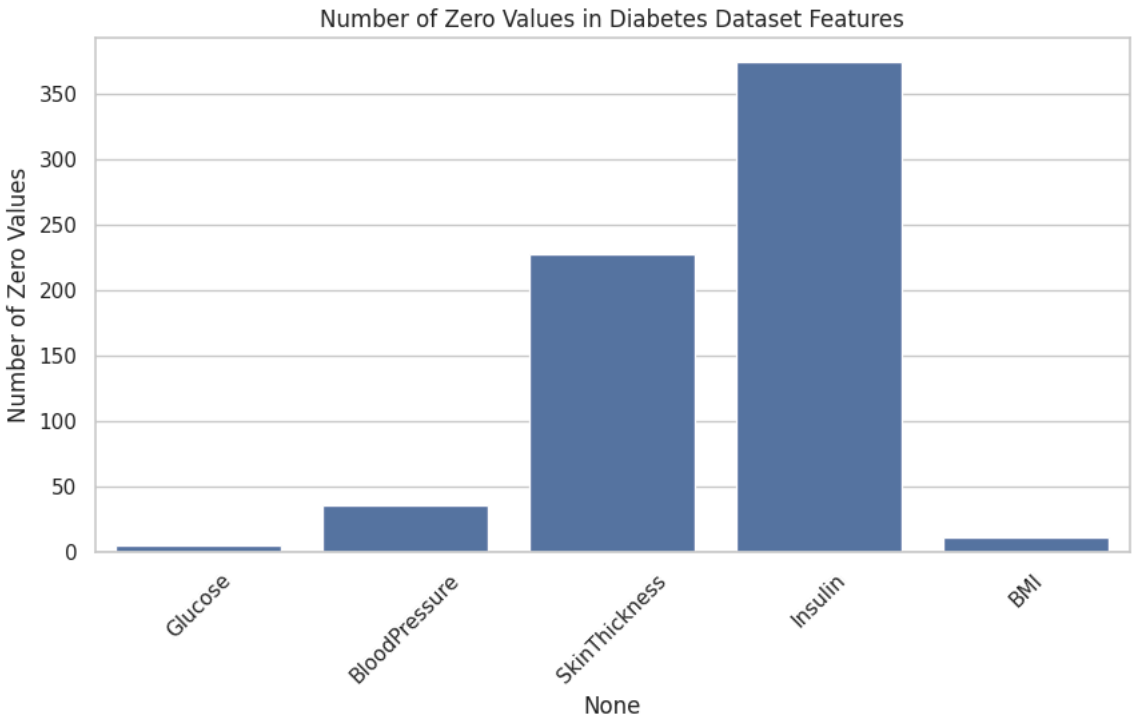
```
Zero values:
Glucose            5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI               11
dtype: int64
```

```
plt.figure(figsize=(10,5))

sns.barplot(x=zero_counts.index,y=zero_counts.values)

plt.xticks(rotation=45)
plt.ylabel("Number of Zero Values")
plt.title("Number of Zero Values in Diabetes Dataset Features")
```

```
Text(0.5, 1.0, 'Number of Zero Values in Diabetes Dataset Features')
```




```
import numpy as np

df_clean = df.copy()

df_clean[zero_columns] = df_clean[zero_columns].replace(0, np.nan)

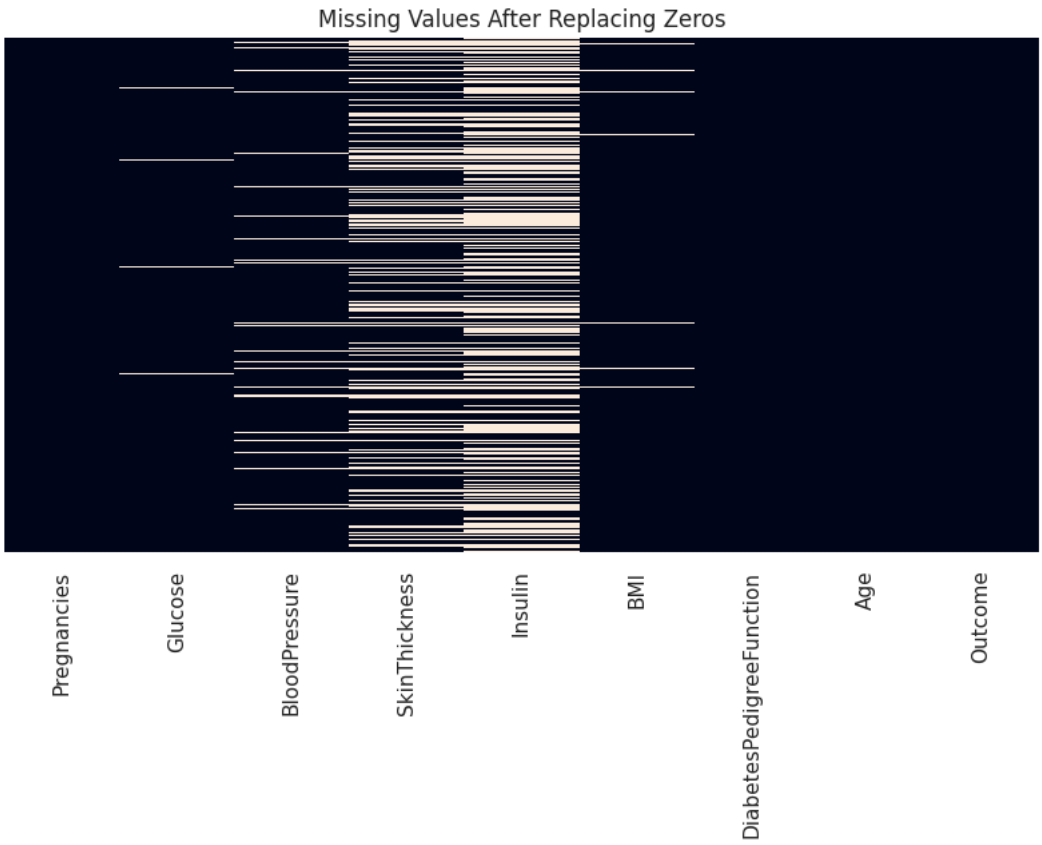
print("Missing values after replacing invalid zeros:")
print(df_clean.isnull().sum())
```

```
Missing values after replacing invalid zeros:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
plt.figure(figsize=(10,5))

sns.heatmap(df_clean.isnull(),cbar=False,yticklabels=False)

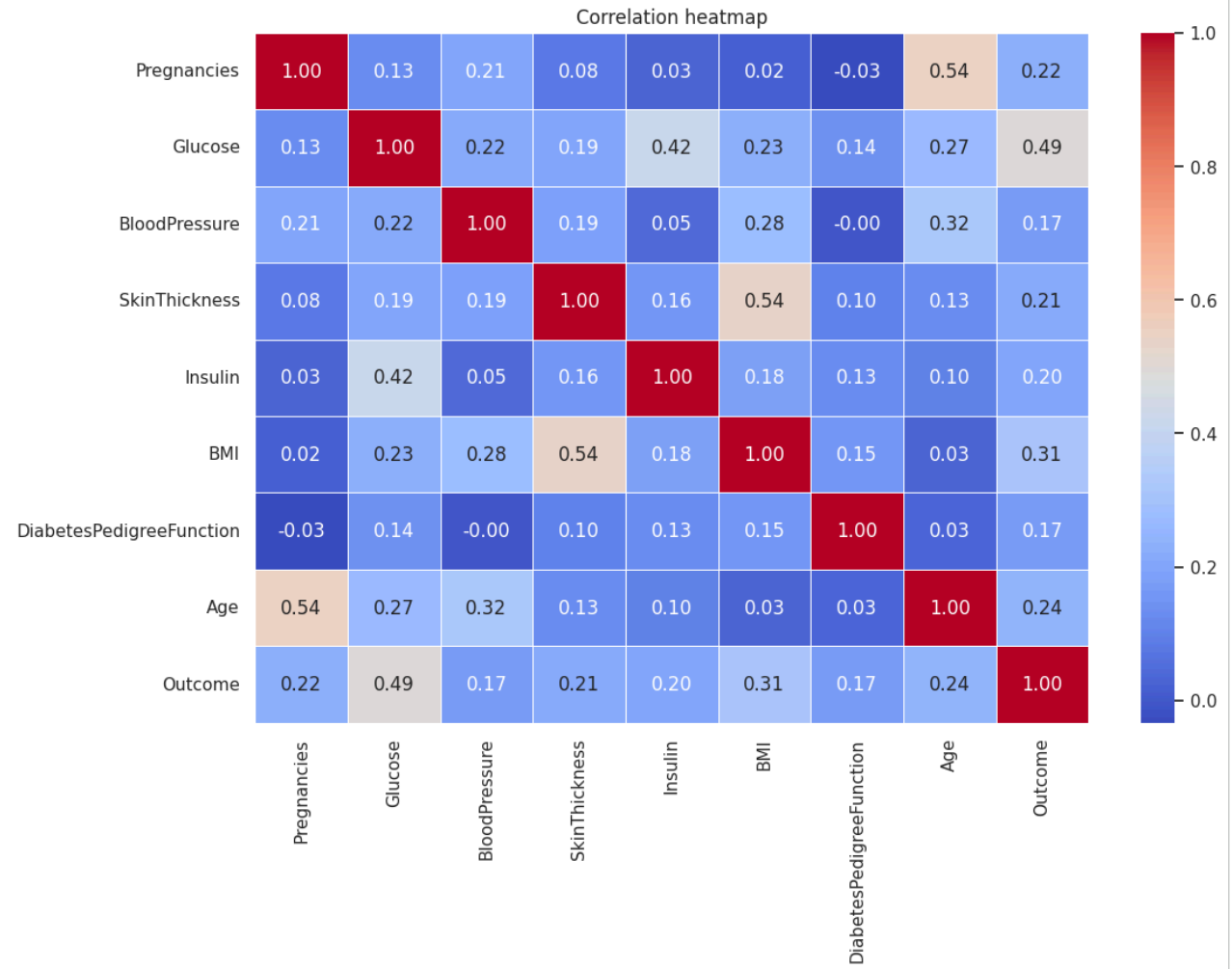
plt.title("Missing Values After Replacing Zeros")
plt.show()
```




```
for col in zero_columns:
   df_clean[col] = df_clean[col].fillna(df_clean[col].median())
print("Missing values after median imputation:")
print(df_clean.isnull().sum())
```

```
Missing values after median imputation:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
plt.figure(figsize=(12,8))
correlation = df_clean.corr()
sns.heatmap(correlation,annot=True,fmt=".2f",cmap="coolwarm",linewidth=0.5)
plt.title("Correlation heatmap")
plt.show()
```

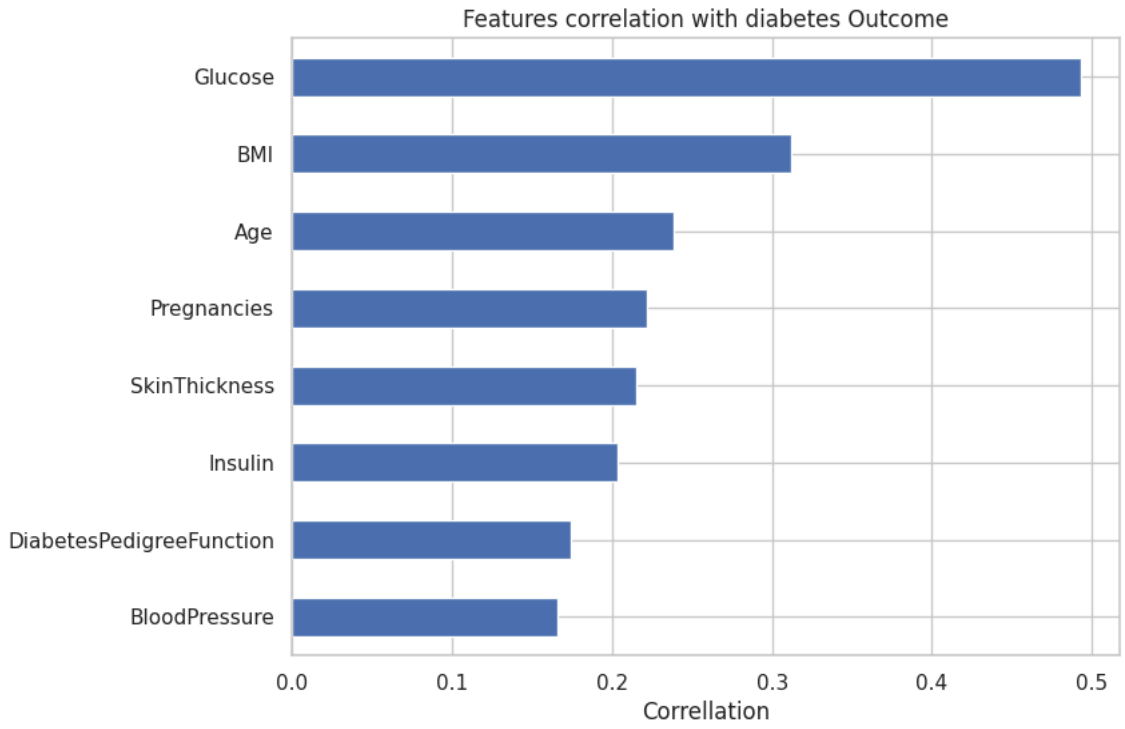



```
target_corr = df_clean.corr()["Outcome"].drop("Outcome").sort_values()

print(target_corr)
```

```
BloodPressure               0.165723
DiabetesPedigreeFunction    0.173844
Insulin                     0.203790
SkinThickness               0.214873
Pregnancies                 0.221898
Age                         0.238356
BMI                         0.312038
Glucose                     0.492782
Name: Outcome, dtype: float64
```

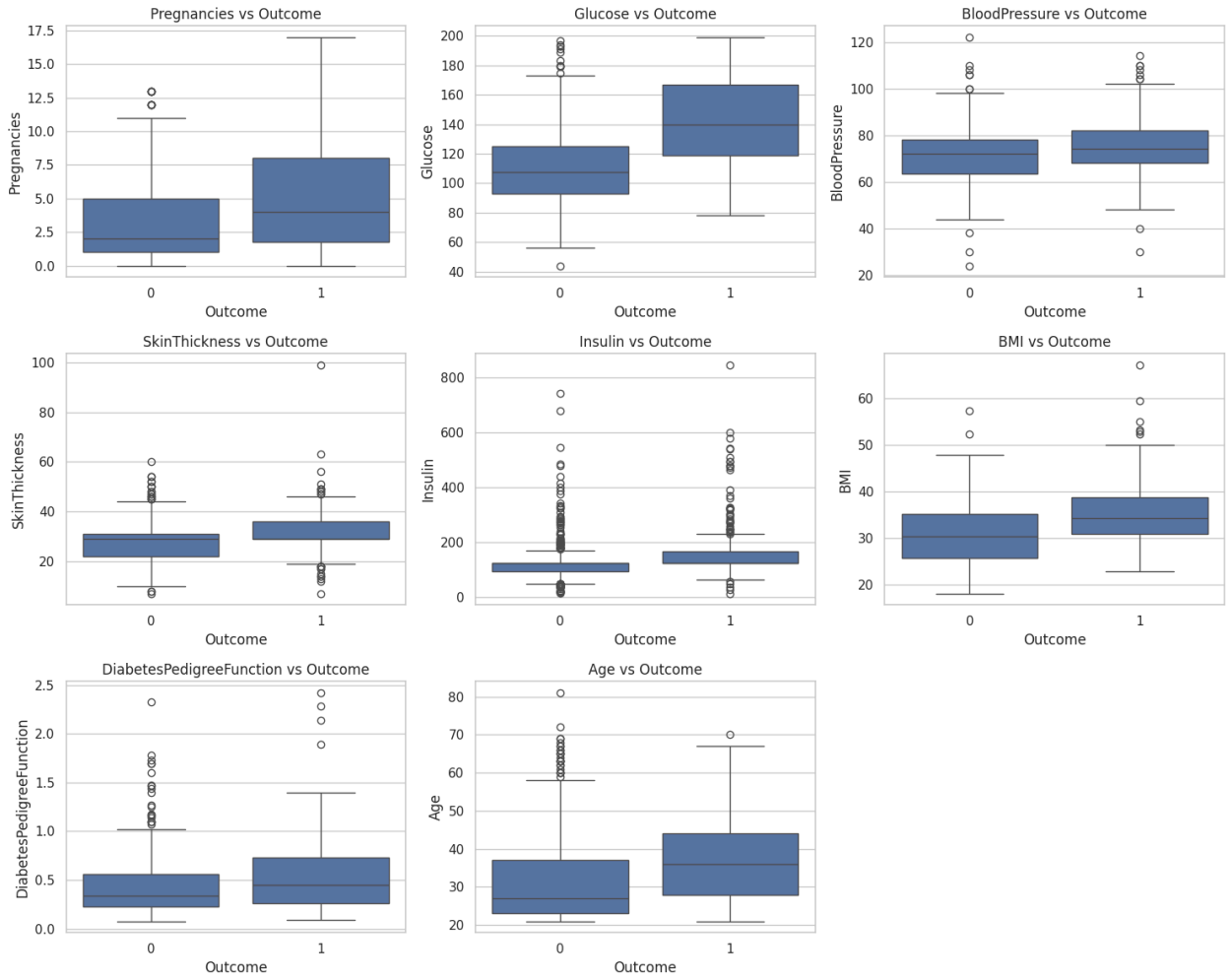
```
plt.figure(figsize=(8,6))
target_corr.plot(kind="barh")
plt.title("Features correlation with diabetes Outcome")
plt.xlabel("Correllation")
plt.show()
```




```
plt.figure(figsize=(15,12))

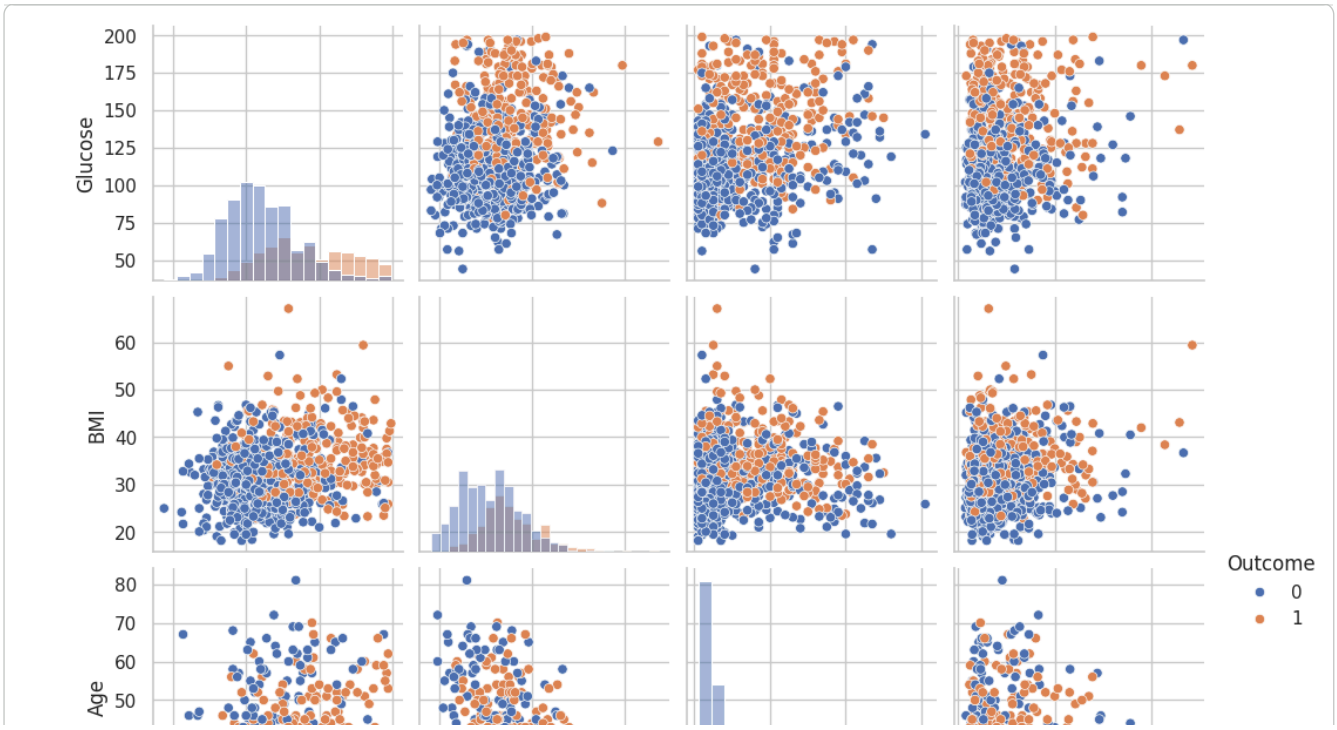
for i, col in enumerate(features, 1):
  plt.subplot(3,3,i)
  sns.boxplot(data=df_clean, x="Outcome", y=col)
  plt.title(f"{col} vs Outcome")

plt.tight_layout()
plt.show()
```




```
selected_features = ["Glucose", "BMI", "Age", "DiabetesPedigreeFunction","Outcome"]
sns.pairplot(df_clean[selected_features], hue="Outcome", diag_kind="hist")

plt.show()
```


Glucose
BMI
Age
Outcome
0
1